

Projet 4OPT201

HELPIQUET Victor, KERJEAN Adrien

Abstract

L'objectif du projet est la gestion d'un portefeuille d'actifs de manière à optimiser ses revenus. Nous nous intéresserons spécifiquement aux problèmes du portefeuille markovien et de Kurtosis. Nous verrons également les bases de différents concepts clés de l'optimisation, comme la relaxation convexe et la programmation quadratique séquentielle pour résoudre notamment les problèmes non convexes. Nous utiliserons plusieurs méthodes implémentées en python pour traiter ces thèmes, en utilisant des bibliothèques comme CVXPY et SCIPY.

1 Pour commencer : explication des grandeurs et sélection du portefeuille

On considère un problème de portefeuille d'actifs classique avec n actifs que l'on possède pour une période donnée. Soit x_i la quantité de l'actif i détenue pendant la période, avec x_i en euros, au prix du début de la période. Soit p_i le prix d'échange relatif de l'action i pendant la période. Le retour global du portefeuille est $r = p^T x$, où nous avons rangé x_i et p_i dans des vecteurs, et la variable d'optimisation est le vecteur portefeuille $x \in \mathbf{R}^n$. Nous adoptons un modèle stochastique pour les variations de prix : $p \in \mathbf{R}^n$ est un vecteur aléatoire de moyenne μ et de covariance Σ . Par conséquent, le rendement r est une variable aléatoire scalaire de moyenne $\mu^T x$ et de variance $x^T \Sigma x$. Le choix du portefeuille x implique un compromis entre la moyenne du rendement et sa variance.

2 Markowitz portfolio problem

Le problème d'optimisation du portefeuille de Markov classique est le programme quadratique suivant,

$$\min_{x \in \mathbf{R}^n} x^T \Sigma x \quad \text{s.t.} \quad \mu^T x \geq r_{\min}, \quad \mathbf{1}^T x = 1, \quad x \geq 0. \quad (1)$$

avec $r_{\min}$ un retour minimal souhaité. Ce problème est convexe.

Ce problème cherche à minimiser la variance pour éclairer ses investissements et ainsi construire un portefeuille d'actifs viable. La contrainte avec μ est celle qui fixe le taux de retour minimal. Le fonctionnement est le suivant : on fixe les poids dans le vecteur x grâce aux données d'entraînement puis on applique ces poids aux données test. On évalue ensuite les résultats que l'on obtient.

3 CVXPY

3.1 Un problème simple de moindres carrés

Intéressons-nous tout d'abord au problème suivant,

$$\min_{x \in \mathbf{R}^n} \frac{1}{2} \|Ax - b\|_2^2 \quad \text{s.t.} \quad 0 \leq x \leq 1. \quad (2)$$

où $A \in \mathbf{R}^{m \times n}$ et $b \in \mathbf{R}^m$ sont générées aléatoirement, pour prendre en main les outils de CVXPY.

(Question 1) Quand on prend $n = 20$, $m = 30$, $A = \text{np.random.randn}(m, n)$ et $b = \text{np.random.randn}(m)$, on obtient une solution cohérente, que l'on montre sur le code Python fourni. Plusieurs choses sont à noter : on obtient bien un vecteur de la taille $n = 20$, et toutes les contraintes sont respectées car on considère par exemple que $-6.7248 \cdot 10^{-21}$, que l'on obtient numériquement vaut 0 en réalité.

(Question 2) On résout maintenant ce problème avec des valeurs de n et m , variant respectivement entre 20 et 2000 et 30 et 3000. On obtient le temps de calcul, en fonction de m et n sur la figure 1.

Sur cette figure, on remarque que le temps de calcul augmente lorsque n et m augmentent. On s'y attendait, car plus n et m sont grands, plus les calculs de la matrice A et du vecteur b seront longs.

(Question 3) Une manière intéressante de résoudre ce problème est d'utiliser la bibliothèque *SCIPY*, et notamment le solveur *SLSQ* de *SCIPY.optimize*.

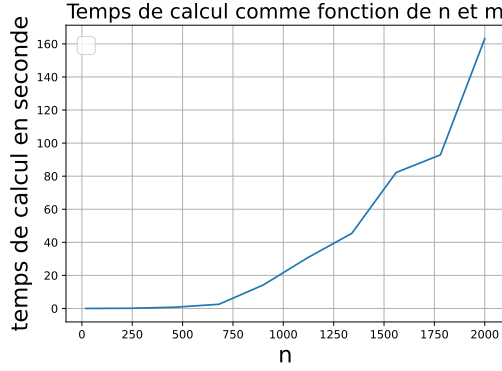


Figure 1: Temps de calcul en fonction de n et m .

Pour différentes valeurs de n et m , variant de la même manière que précédemment, le temps de calcul était très grand (plus d’une heure sans résultat). Ainsi, le temps de calcul est bien plus grand pour cette méthode, qui est donc pour ce problème bien moins efficace que CVXPY. On pouvait s’y attendre car CVXPY est spécialisé pour ce type de problème. On s’attendait à avoir la même allure de graphe, ainsi qu’un temps plus long (mais pas aussi long tout de même !). Cette case est commentée dans notre code, pour permettre à l’utilisateur de ne pas y passer trop de temps, mais d’apprécier tout de même le travail fourni.

3.2 Résoudre le problème du portefeuille markovien

Nous allons désormais utiliser *CVXPY* pour résoudre le Problème (1). Plus précisément, en utilisant les fonctions fournies que sont `download-finance-data(n-assets=3)`, `compute-moments(Ytrain)` et en entrainant les meilleurs poids sur quatre ans (de 2016 à 2019), nous allons tester notre résolution sur l’année 2020.

(**Question 4**) Pour un retour minimal $r_{min} = 1.6/252$, tel que notre solution optimale ait un meilleur taux que le Livret A, on code la fonction `markovitz-portfolio(mu, sigma)`, qui résout le problème et retourne le vecteur x optimal. Pour 3 actifs, on obtient par exemple $x = [0.23014872 \quad 0.51515729 \quad 0.25469399]^T$. Les poids somment bien à 1 (tout est investi).

(**Question 5**) Pour mieux comprendre ce résultat, on va utiliser les métriques de variance, retour (en pourcentage), et d’écart-type, pour les données d’entraînement, et pour les données de test. On utilise ces fonctions pour 3 et 20 actifs et on obtient les résultats rangés dans le tableau suivant :

Données	Nombre d’actifs	Retour (%)	Variance	Écart-type
Entraînement	3	19.724686	1.079553	16.493860
Entraînement	20	12.902768	0.432029	10.434140
Test	3	0.360059	5.494726	37.211169
Test	20	5.404801	4.152703	32.349360

Table 1: Résultats pour 3 et 20 actifs sur les données d’entraînement et de test.

On observe qu’il y a une grande différence entre les résultats issus des données de test et des données d’entraînement. Ici, le portefeuille à 3 actifs se comporte très mal sur les données de test : on obtient un rendement quasi nul ainsi qu’une volatilité énorme. Cela est lié au fait qu’il est très peu diversifié. Le portefeuille à 20 actifs a lui aussi une volatilité élevée mais moins que celui à 3 actifs, et obtient un rendement test meilleur, bien que son rendement pour les données d’entraînement soit moins bon. Cela paraît cohérent : lorsque l’on veut limiter la variance, on se diversifie davantage pour diluer les risques.

3.3 Variations probabilistes

Considérons maintenant une contrainte de risque de perte de la forme $\text{prob}(r \leq \alpha) \leq \beta$, où α est un niveau de rendement indésirable donné (par exemple une perte importante) et β une probabilité maximale donnée. Nous exprimons cette contrainte à l’aide de la fonction de répartition cumulative Φ . L’inégalité ci-dessus est équivalente à $\mu^T x + \Phi^{-1}(\beta) \|\Sigma^{1/2} x\|_2 \geq \alpha$.

Le problème devient alors

$$\begin{aligned} \min_{x \in \mathbf{R}^n} \quad & x^T \Sigma x \\ \text{s.t.} \quad & \mu^T x + \Phi^{-1}(\beta) \|\Sigma^{1/2} x\|_2^2 \geq \alpha, \\ & x \geq 0, \quad \mathbf{1}^T x = 1. \end{aligned} \quad (3)$$

(**Question 6**) On utilise $\beta = 0.49$ et $\alpha = 1.6/252$. on obtient le tableau de résultats suivant :

Période / Jeu	Poids du portefeuille	Retour (%)	Écart-type	Variance
2016–2019 (train)	[0.23195316, 0.51986342, 0.24818342]	19.852618	16.494789	1.079675
2020 (test)	[0.23195316, 0.51986342, 0.24818342]	0.332174	37.337568	5.532119

Table 2: Performance du portefeuille à 3 actifs sur les données d’entraînement (2016–2019) et de test (2020).

Les résultats sont moins bons que ceux du problème (1) : le retour sur les données d’entraînement est légèrement plus faible ici bien que le retour sur les données de test est plus élevé. De plus, Le vecteur de poids [0.2319, 0.5199, 0.2482] montre un portefeuille assez concentré sur le deuxième actif.

En entraînement, ce portefeuille donne un rendement d’environ 19,85 pourcents avec un écart-type de 16,49 pourcents, très proche de ce qu’on obtenait pour trois actifs dans le problème (1) (même ordre de grandeur de rendement et de volatilité). En test, en revanche, le rendement chute à 0,33 pourcent pour une volatilité énorme (37,34 pourcents) et une variance de 5,53, alors que dans le problème (1), on avait un rendement test plus élevé. Ainsi, la contrainte probabiliste qu’on a ajoutée n’améliore pas le comportement sur les données test par rapport au problème (1) : on a toujours un rendement test très faible, ce qui suggère un fort sur-ajustement aux données d’entraînement.

En augmentant le nombre d’actifs et en réduisant β , on obtient les résultats compilés dans le tableau suivant, dans lequel les données obtenues en faisant varier le nombre d’actifs sont sous le nom de "Markovien" :

Type	# actifs	β	Retour train (%)	Var. train	Retour test (%)	Var. test
Markovien	3	0.49	18.41	0.8835	7.45	4.676
Markovien	4	0.49	17.18	0.6749	3.84	6.505
Markovien	5	0.49	15.33	0.6171	4.63	5.076
Markovien	6	0.49	15.35	0.6159	5.31	5.047
Markovien	7	0.49	14.96	0.6001	6.57	4.952
Markovien	8	0.49	15.72	0.5841	5.81	5.283
Markovien	9	0.49	15.27	0.5792	5.57	5.299
Probabiliste	3	0.490	19.85	1.0797	0.33	5.532
Probabiliste	3	0.489	21.50	1.1033	-0.02	6.021
Probabiliste	3	0.488	23.21	1.1719	-0.36	6.533
Probabiliste	3	0.487		infeasible (aucune solution)		
Probabiliste	3	0.486		infeasible (aucune solution)		

Table 3: Résumé des performances selon le nombre d’actifs et selon la valeur de β pour le portefeuille probabiliste à 3 actifs.

On remarque que tant qu’on continue à diminuer β et qu’on peut calculer le rendement, plus ce dernier est bon pour les données d’entraînement. Il passe de 19,85 pourcents pour $\beta = 0.49$ à 23,21 pourcents de retour pour $\beta = 0.488$, tout en gardant une variance et un écart-type raisonnables. Néanmoins, pour les données de test, on obtient un rendement négatif et une augmentation de la variance lorsque β diminue, ce qui montre que durcir la contrainte β affaiblit encore plus les résultats du portefeuille, et sur-entraîne encore plus le modèle.

Lorsqu’on modifie le nombre d’actifs, on retombe sur les constatations précédentes : à β fixé, l’augmentation du nombre d’actif mène à une stabilisation des performances du portefeuille.

Pour ce qui est de la dépendance entre la variance et l’augmentation du nombre d’actifs pour cette résolution, on peut s’appuyer sur la figure 2, qui représente la variance en fonction du nombre d’actifs pour les données d’entraînement. On voit bien qu’on arrive aux mêmes conclusions qu’en première partie pour les données d’entraînement : plus on augmente le nombre d’actifs, plus la variance diminue. Cela est moins vérifié pour les données test, qui atteignent une variance optimale pour $n = 8$, ce qui est contre-intuitif et montre sûrement le sur-entraînement sur les données de 2016 à 2019.

On peut néanmoins dire que l’année 2020 était l’année COVID, et donc ne ressemblait à aucune précédente, ce qui fragilise les hypothèses d’un modèle markovien qui implicitement suppose que le futur ressemble au passé. Donc l’entraînement sur les années précédentes ne pouvait sûrement pas bien prédire les performances.

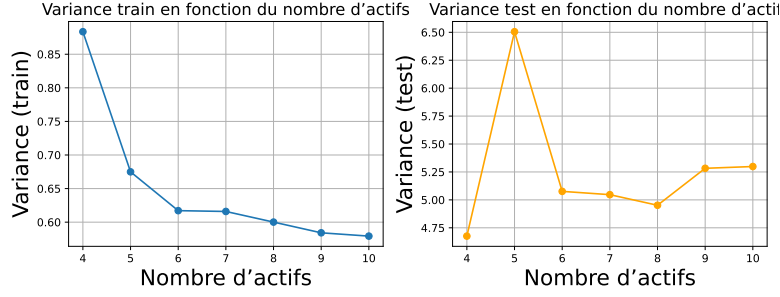


Figure 2: Variance en fonction du nombre d'actifs n .

(**Question 7**) Si l'on utilise pour les données test les années 2020 et 2021, et non plus seulement 2020, qui était une année particulière à cause du COVID, on obtient, pour $n = 3$ et le β standard les résultats du tableau 4.

Période / Jeu	Poids du portefeuille	Retour (%)	Écart-type	Variance
2016–2019 (train)	[0.23195316, 0.51986342, 0.24818342]	19.852618	16.494789	1.079675
2020–2021 (test)	[0.23195316, 0.51986342, 0.24818342]	4.046558	28.403955	3.201527

Table 4: Performance du portefeuille à 3 actifs sur les données d'entraînement (2016–2019) et de test (2020–2021).

Même s'il reste faible, le retour est de 4.04 pourcents, ce qui est mieux que lorsque l'on considèrerait uniquement 2020. De plus, l'écart-type est de 28.4 sur les données test, donc près de 10 points plus faibles qu'avant, et la variance est elle aussi plus faible d'environ 2 points. Cela semble logique : 2021 ressemble beaucoup plus aux années des données d'entraînement que 2020. On obtient donc, en combinant 2020 et 2021, de meilleurs résultats qu'en considérant 2020 seule. Si on ne considère que 2021, on obtient en effet de bien meilleurs résultats (9.032709 pourcents de retour pour trois assets, pour un retour d'entraînement à 19.852628 avec une volatilité faible, car il s'agit d'une année ressemblant bien plus aux années d'entraînement).

3.4 Représentations graphiques

(**Question 8**) Pour trois actifs, les données des séries temporelles dans la phase d'entraînement et de test sont sur les figures 3 et 4.

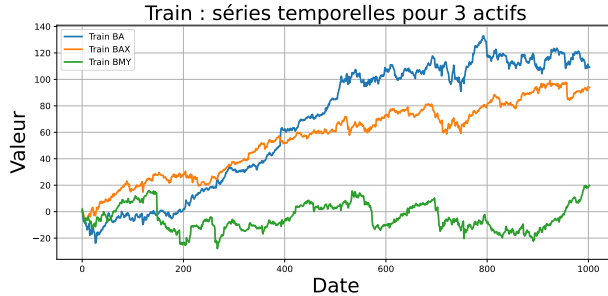


Figure 3: données des séries temporelles pour 3 actifs (phase d'entraînement)

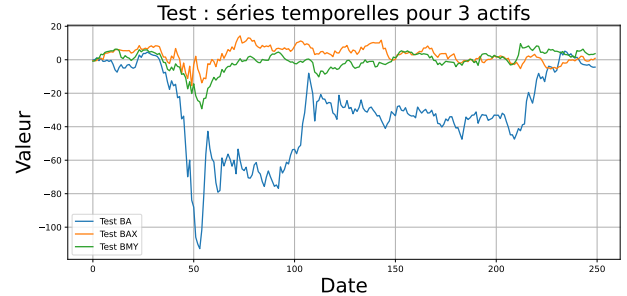


Figure 4: données des séries temporelles pour 3 actifs (phase de test)

(**Question 9**) On réalise ensuite une analyse de Monte-Carlo, et on obtient les figures 5 et 6 dans le cas $n_{assets} = 3$. On ne représente pas le cas $n_{assets} = 20$ pour des raisons de place, mais le lecteur pourra les trouver dans notre code s'il le souhaite.

Dans le cas $n_{assets} = 3$, pour les données d'entraînement, le nuage de points forme une frontière efficiente : les portefeuilles situés en haut à gauche ont un rendement élevé pour un risque plus faible, ceux en bas à droite ont peu de rendement pour beaucoup de risque. La solution optimisée est proche de l'enveloppe supérieure du nuage, dans la zone de rendement élevé et de volatilité modérée : l'algorithme trouve bien un portefeuille quasi-efficient sur les données d'entraînement, donc la solution retournée fonctionne bien sur ces données.

Néanmoins, pour les données de test, Le nuage est descendu, étiré vers la droite : les rendements sont en moyenne plus faibles, les volatilités sont plus fortes qu'avant. L'étoile rouge se retrouve vers le milieu bas de ce nuage :

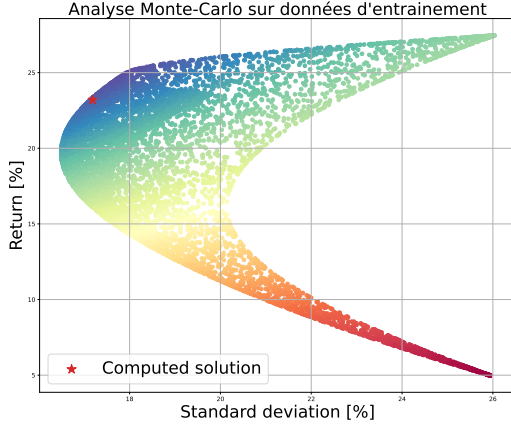


Figure 5: Analyse de Monte-Carlo pour 3 actifs sur données d'entraînement

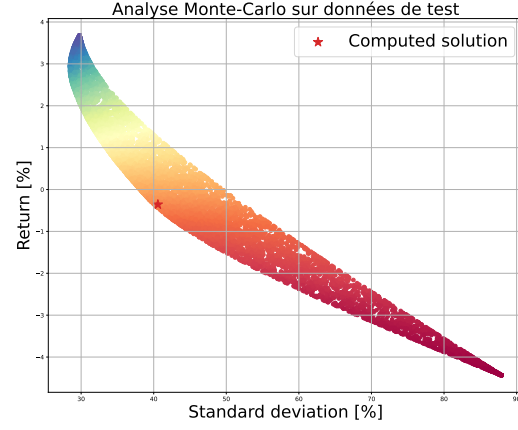


Figure 6: Analyse de Monte-Carlo pour 3 actifs sur données de test

pour le niveau de risque qu'elle porte, son rendement n'est plus du tout exceptionnel.

Le portefeuille optimal sur 2016–2019 n'est plus proche de la frontière efficiente sur 2020–2021 : il est sur-ajusté au régime pré-Covid. Il perd donc son avantage quand la distribution des rendements change. Ainsi ces figures illustrent à la fois l'efficacité de l'optimisation moyenne-variance en régime stationnaire sur les données d'entraînement et sa forte sensibilité aux changements de régime, comme par exemple une année COVID.

4 Kurtosis

On utilise la Kurtosis lorsque minimiser la variance n'est pas suffisant pour obtenir un bon portefeuille. Nous nous intéressons désormais à la formulation du problème

$$\begin{aligned} & \min_{x \in \mathbf{R}^n, X \in S(R^n), z \in R^{n(n+1)/2}, g \in R} g \\ \text{s.t. } & X = xx^\top, \quad \mu^\top x \geq r_{\min}, \quad z = L_2 \text{vec}(X), \\ & \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} z \right\|_2 \leq g, \quad \sum_{i=1}^n x_i = 1, \quad x \geq 0, \end{aligned} \quad (4)$$

où L_2, S_2, Σ_4 sont spécifiques au problème.

(**Question 10**) Ce problème n'est pas convexe car certaines contraintes ne sont pas convexes. Précisément, la contrainte $X = xx^\top$ n'est pas convexe. En effet, comme $x \in R^n, xx^\top \in R^{n \times n}$ est de rang 1.

Contre-exemple de non-convexité de l'ensemble $\mathcal{S} = \{(x, X) \mid X = xx^\top\}$

Considérons $n = 2$ et deux points dans $\mathcal{S}$:

Considérons deux vecteurs $x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ tel que $X_1 = x_1 x_1^\top = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ (**rang 1**) et $x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ tel que $X_2 = x_2 x_2^\top = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ (**rang 1**). La combinaison convexe pour $t = 0.5$ donne $X_m = 0.5X_1 + 0.5X_2 = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$. Donc le rang de X_m vaut $2 \neq 1$, donc $X_m \notin \mathcal{S}$.

L'ensemble n'est pas convexe car les combinaisons convexes ne préservent pas le rang 1. Donc le problème n'est pas convexe.

5 Programmation séquentielle quadratique

5.1 Une problème non convexe simple

Considérons un problème non convexe simple de la forme

$$\min_{x \in \mathbf{R}^n} \frac{1}{2} \|Ax - b\|_2^2 \quad \text{s.t.} \quad 0 \leq x \leq 1, \quad \|x\|_2^2 \geq \frac{1}{2}, \quad (5)$$

où $A \in \mathbf{R}^{m \times n}$ et $b \in \mathbf{R}^m$ sont générés aléatoirement.

(**Question 11**) Après avoir codé un solveur SQP, et en utilisant CVXPY pour chaque sous-problème convexe obtenu, et en linéarisant la contrainte quadratique, on obtient les quantités représentées sur la figure 7. On fait la même chose pour la (**Question 12**) avec SCIPY.optimize.

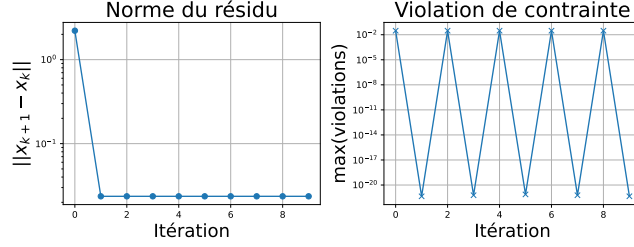


Figure 7: Norme du résidu et estimation de la violation de la contrainte avec CVXPY

On observe que le résidu décroît rapidement puis devient quasiment constant et très petit, donc l'algorithme a convergé numériquement. La violation de contrainte oscille entre 10^{-2} et la valeur machine 10^{-22} , ce qui correspond au fait que la contrainte de norme est parfois active, parfois inactive au voisinage de la frontière. Les contraintes finales sont satisfaites. La fonction objectif f admet donc un minimum de $f(x) = 9.918961$.

Concernant la comparaison de la (**Question 12**), le SLSQP converge en 19 itérations, vers une solution égale à $f(x_{SLSQP}) = 9.918843$ qui satisfait toutes les contraintes. On observe aussi que le temps de calcul est plus long pour la méthode SCP.

5.2 Problème de portefeuille kurtosis

On considère le problème de perturbation

$$\begin{aligned}
 & \min_{\delta x \in \mathbf{R}^n, \delta X \in \mathcal{S}(R^n), \delta z \in R^{n(n+1)/2}, \delta g \in R} g^k + \delta g \\
 \text{s.t. } & X^k + \delta X = x^k x^{k\top} + 2x^k(\delta x)^\top, z^k + \delta z = L_2 \text{vec}(X^k + \delta X), \\
 & \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} (z^k + \delta z) \right\|_2 \leq g^k + \delta g, \quad \sum_{i=1}^n x_i^k + \delta x_i = 1, \quad X^k + \delta X \succeq 0 \\
 & x^k + \delta x \geq 0, \quad \mu^\top (x^k + \delta x) \geq r_{\min}.
 \end{aligned} \tag{6}$$

(**Question 13**) On va maintenant utiliser une succession de problèmes de perturbation, et donc utiliser la méthode SCP pour résoudre le problème de kurtosis du portefeuille. Ainsi, après implémentation, on obtient les figures 8 et 9.

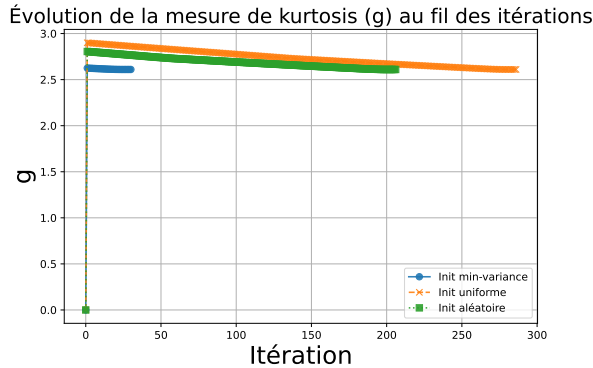


Figure 8: Convergence de l'algorithme SCP pour différentes conditions initiales.

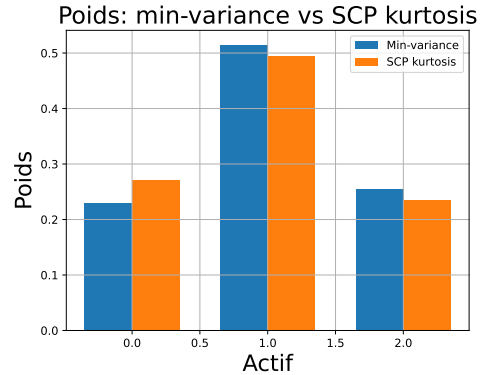


Figure 9: Poids des actifs (cas 3 actifs) pour les deux méthodes étudiées.

Ainsi, pour différentes conditions initiales (min-variance, uniforme, aléatoire), l'algorithme converge vers la même valeur finale de kurtosis $g = 2,61$. Il est à noter que l'initialisation min-variance converge en très peu d'itérations (30), alors que les initialisations uniforme et aléatoire descendent progressivement mais finissent au même niveau.

Cela montre que l'algorithme SCP est robuste aux choix d'initialisation et qu'il trouve un même optimum. Enfin, le graphique « Poids: min-variance vs SCP kurtosis » montre que les poids optimisés pour la kurtosis restent proches de la solution min-variance, avec de légers ajustements sur chaque actif. Cela signifie que l'algorithme SCP n'a pas modifié significativement l'allocation des actifs de min-variance pour atteindre l'objectif. **(Question 14)** On a ensuite résolu avec SCIPY.optimize et le solveur SLSQP et on observe que ces deux méthodes permettent d'obtenir la même solution pour le problème (4). De plus, le temps de calcul est bien plus élevé pour SCP que pour SLSQP, ce qui est normal car SCP est un algorithme de résolution général, là où SLSQP ne s'applique qu'aux problèmes convexes. Pour aller plus loin, on peut s'appuyer sur la figure 10.

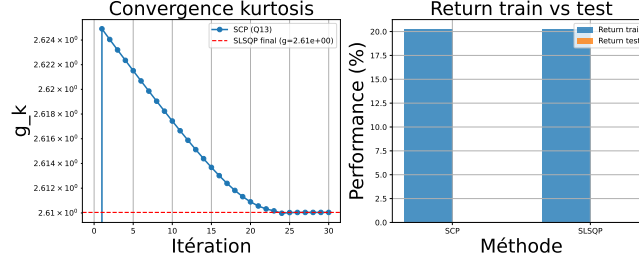


Figure 10: Convergence de g et comparaison des retours sur les données d'entraînement et de test.

Pour la figure de droite, on observe que les performances du SCP et du SLSQP sont identiques. Les performances sur les données de test sont en revanche médiocres, ce qui peut être dû à un sur-apprentissage, qui mène à des résultats peu convaincants. La figure de gauche nous montre bien la convergence vers la même solution.

(Question 15) On mène en complément une analyse de Monte-Carlo. On met les résultats pour les deux méthodes sur le même graphes, un pour chaque type de données. On obtient ainsi les figures 11 et 12.

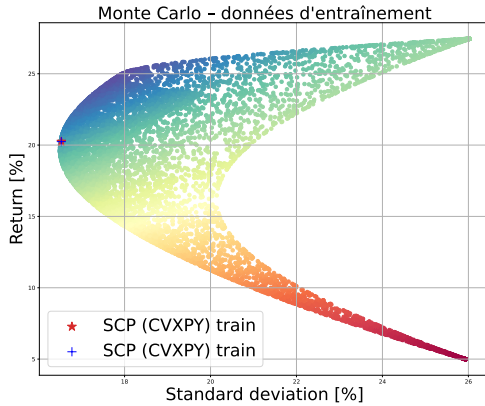


Figure 11: Analyse de Monte-Carlo pour comparer les méthodes SCP et SLSQP (données d'entraînement)

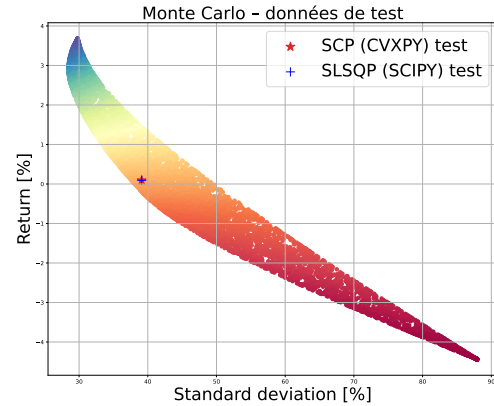


Figure 12: Analyse de Monte-Carlo pour comparer les méthodes SCP et SLSQP (données de test)

On voit bien que les points se superposent. Les deux méthodes convergent donc vers la même solution et donc le même portefeuille.

6 Relaxation Convexe

On s'intéresse désormais à la relaxation convexe du problème (4), donnée par

$$\begin{aligned}
 & \min_{x \in \mathbf{R}^n, X \in \mathcal{S}(\mathbf{R}^n), z \in \mathbf{R}^{n(n+1)/2}, g \in \mathbf{R}} g \\
 \text{s.t. } & \begin{bmatrix} X & x \\ x^\top & 1 \end{bmatrix} \succeq 0, z = L_2 \text{vec}(X), \quad \mu^\top x \geq r_{\min}, \\
 & \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} z \right\|_2 \leq g, \sum_{i=1}^n x_i = 1, \quad x \geq 0.
 \end{aligned} \tag{7}$$

(**Question 16**) Hormis la première contrainte, toutes les autres contraintes et la fonction objectif sont similaires. Il y a donc un travail à faire dans la compréhension de la première condition de (7).

Dans le problème de base (4), on a certaines contraintes qui ne sont pas linéaires. Le problème (7) est donc le problème résultant de la linéarisation de la contrainte quadratique $X = xx^T$ (relaxation convexe), et est donc convexe. Ainsi, (7) est un problème proche de (4), mais il ne s'agit pas du problème (4) tout à fait. On ne résout donc pas le problème (4) en résolvant le problème (7). Pour s'approcher de la résolution de (4), il faudrait résoudre (6) en boucle.

La valeur optimale de (7) est ni garantie plus petite ni plus grande globalement que celle de (4) : cela dépend si on a construit une sous- ou sur-approximation : Si (7) est une sous-estimation, sa valeur optimale est inférieure à celle de (4). Si (7) est une sur-approximation, sa valeur est supérieure à celle de (4). Dans le cas SCP, on majore l'objectif et les contraintes, donc on a une solution supérieure ou égale à celle de (4).

(**Question 17**) On va maintenant résoudre (7) avec CVXPY. On compare avec les résultats de Q14. Les résultats comparatifs sont rangés dans le tableau 5.

n_{assets}	Méthode	mean _{tr}	var _{tr}
3	Min-variance	0.07827	1.07848
3	Kurtosis problème (10)	0.08032	1.08305
10	Min-variance	0.06061	0.57865
10	Kurtosis problème (10)	0.05765	0.58711
23	Min-variance	0.05145	0.42687
23	Kurtosis problème (10)	0.04415	0.44618

Table 5: Résultats train pour les portefeuilles min-variance et problème de kurtosis (6).

Avec SLSQP, on obtenait un temps de résolution de 0.007s pour $n_{\text{assets}} = 3$. Avec SCP, la **question 13** fournit un temps de calcul moyen de 44,2 secondes, bien supérieur au temps de calcul du solveur SLSQP donc. Pour le Problème (7), on observe ici un temps de 0.008s, ce qui est cohérent avec l'ordre de grandeur des temps obtenus précédemment pour cette méthode. De plus, le temps de calcul augmente avec la dimension, mais reste raisonnable pour les tailles testées (0.4s pour $n_{\text{assets}} = 23$ au maximum).

On observe que $|X - xx^T|$ est très faible pour chaque n_{assets} testé ($\sim 10^{-6}$), ce qui indique que la relaxation est presque exacte pour nos données : les variables optimales vérifient quasiment la contrainte non convexe initiale. La kurtosis minimale atteignable décroît avec n , ce qui est normal car il y a plus de diversification dans le portefeuille. L'algorithme retrouve donc un point quasi réalisable du problème original. On aurait pu avoir une erreur bien plus grande si la relaxation était moins précise. Pour $n = 3, 10, 23$, le portefeuille obtenu via la relaxation convexe (7) présente un rendement moyen légèrement supérieur au min-variance, pour une variance très proche. De plus, la valeur de g^* obtenue avec (7) est inférieure à celle trouvée par SCP/SLSQP en Q13/14, cela montre l'intérêt pratique du problème (6): on améliore le comportement sans détériorer les résultats de rendement et variance.

7 Conclusions

(**Question 18**) Ce projet a permis de comparer des méthodes d'optimisation classiques (CVXPY, SLSQP) et séquentielles (SCP) sur des problèmes de portefeuille allant du modèle de Markowitz à des formulations intégrant la kurtosis. Sur les problèmes convexes, CVXPY se révèle à la fois plus rapide et plus fiable que SLSQP. L'introduction de la kurtosis montre qu'il est possible de mieux contrôler le risque, au-delà de la simple variance.

En particulier, la relaxation convexe du problème de kurtosis (10) fournit numériquement une solution presque exacte et des performances rendement-variance proches de Markowitz. Enfin, l'étude de l'impact du nombre d'actifs confirme l'effet de diversification sur la kurtosis minimale atteignable. Enfin, on a aussi vu comment résoudre des problèmes de portefeuille même si l'hypothèse de convexité est mise en défaut.

(**Bonus**) Il est très difficile d'utiliser le ML car le problème est non-stationnaire : les statistiques (μ, Σ) changent dans le temps, donc un modèle entraîné en 2016-2019 peut être obsolète en 2020, surtout avec le COVID qui a brutalement changé les corrélations. Il faut aussi faire attention au surapprentissage. Enfin, le ML coûte cher énergétiquement, donc il faut pouvoir justifier que son utilisation va nous faire gagner plus que ce qu'elle nous coûte. Pour conclure, cette difficulté montre définitivement une chose : les marchés restent partiellement imprévisibles par nature.